设备总览后端流程与部署指南

1. 目标

本文档用于指导后端提前部署，并与现有前端 `设备总览` 页面完成联调。

本次范围包括：

- Excel 数据导入流程
- 数据标准化与入库方案
- 设备总览相关接口设计
- 前后端联调约定
- NestJS + PostgreSQL 部署方法
- 环境变量与运行配置

2. 当前真实 Excel 字段

当前首批真实 Excel 数据源字段如下：

- `采购`
- `运营`
- `设备在线情况`
- `设备提供`
- `供应商`
- `计费方式`
- `账户编码`
- `5G设备`
- `10G设备`
- `2G设备`
- `1G小主机`
- `报废设备`

这些字段中：

- `采购`、`运营`、`设备在线情况`、`设备提供`、`供应商`、`计费方式`、`账户编码` 属于维度字段
- `5G设备`、`10G设备`、`2G设备`、`1G小主机`、`报废设备` 属于设备数量字段

3. 推荐的数据处理思路

当前 Excel 更像“汇总行数据”，而不是“一台设备一行”的明细台账。

因此后端不要直接把 Excel 原样用于图表查询，而应采用两层表模型：

- 原始层：保存 Excel 原始记录
- 标准层：将不同设备列拆成统一的设备事实记录

4. 推荐数据库表结构

4.1 原始导入表

表名建议：`import_device_raw`

用途：

- 保存 Excel 原始每一行
- 支持追溯
- 支持重新映射与重新入库

建议字段：

```
id uuid primary key
batch_id uuid not null
source_file_name varchar(255) not null
sheet_name varchar(120)
purchase text
operation text
online_status text
device_provider text
vendor text
billing_mode text
account_code text
device_5g integer default 0
device_10g integer default 0
device_2g integer default 0
device_1g integer default 0
scrapped integer default 0
raw_payload jsonb
created_at timestamptz default now()
updated_at timestamptz default now()
```

4.2 标准化事实表

表名建议：`device_inventory_fact`

用途：

- 用于设备总览页面统计
- 用于按设备类型、供应商、运营等维度聚合

建议字段：

```
id uuid primary key
batch_id uuid not null
purchase text
operation text
online_status text
device_provider text
vendor text
billing_mode text
account_code text
device_type varchar(50) not null
device_count integer not null default 0
created_at timestamptz default now()
updated_at timestamptz default now()
```

建议 `device_type` 允许值如下：

- `5G设备`
- `10G设备`
- `2G设备`
- `1G小主机`
- `报废设备`

5. Excel 导入与标准化流程

建议流程如下：

1. 上传 Excel 文件 2. 解析 Sheet 与表头 3. 校验是否包含必要字段 4. 将原始记录写入 `import_device_raw` 5. 将每行中的 `5G设备`、`10G设备`、`2G设备`、`1G小主机`、`报废设备` 拆成多行 6. 写入 `device_inventory_fact` 7. 提供 Dashboard 和设备总览接口查询

5.1 标准化示例

如果某一行 Excel 为：

```
运营 = 华东运营
设备在线情况 = 在线
供应商 = 供应商A
计费方式 = 包月
账户编码 = ACC-1001
5G设备 = 12
10G设备 = 8
2G设备 = 4
1G小主机 = 6
报废设备 = 1
```

则写入 `device_inventory_fact` 时应拆成：

```
device_type=5G设备, device_count=12
device_type=10G设备, device_count=8
device_type=2G设备, device_count=4
device_type=1G小主机, device_count=6
device_type=报废设备, device_count=1
```

这样后续聚合最简单，SQL 也更稳定。

6. 后端接口清单

前端当前已经预留以下接口：

- `GET /api/dashboard/device-overview/summary`
- `GET /api/dashboard/device-overview/distribution`
- `GET /api/dashboard/device-overview/vendors`
- `GET /api/dashboard/device-overview/table`

建议统一支持以下查询参数：

- `operation`
- `vendor`
- `billingMode`
- `onlineStatus`
- `accountCode`
- `page`
- `pageSize`

7. 接口返回格式

7.1 汇总接口

`GET /api/dashboard/device-overview/summary`

返回示例：

```
{
  "totalOnlineDevices": 1326,
  "totalScrappedDevices": 74,
  "totalVendors": 6,
  "totalAccounts": 14,
  "deviceStats": [
    { "type": "5G设备", "count": 486, "onlineCount": 452, "utilization": 93 },
    { "type": "10G设备", "count": 332, "onlineCount": 316, "utilization": 95 },
    { "type": "2G设备", "count": 241, "onlineCount": 218, "utilization": 90 },
    { "type": "1G小主机", "count": 193, "onlineCount": 182, "utilization": 94 },
    { "type": "报废设备", "count": 74, "onlineCount": 0, "utilization": 0 }
  ]
}
```

7.2 在线分布接口

`GET /api/dashboard/device-overview/distribution`

返回示例：

```
{
  "items": [
    { "type": "5G设备", "count": 452 },
    { "type": "10G设备", "count": 316 },
    { "type": "2G设备", "count": 218 },
    { "type": "1G小主机", "count": 182 },
    { "type": "报废设备", "count": 74 }
  ]
}
```

7.3 供应商排行接口

`GET /api/dashboard/device-overview/vendors`

返回示例：

```
{
  "items": [
    { "vendor": "供应商A", "count": 328 },
    { "vendor": "供应商B", "count": 274 },
    { "vendor": "供应商C", "count": 243 }
  ]
}
```

7.4 明细表格接口

`GET /api/dashboard/device-overview/table`

返回示例：

```
{
  "total": 4,
  "items": [
    {
      "operation": "华东运营",
      "onlineStatus": "在线",
      "provider": "自有",
      "vendor": "供应商A",
      "billingMode": "包月",
      "accountCode": "ACC-1001",
      "device5g": 32,
      "device10g": 14,
      "device2g": 6,
      "device1g": 8,
      "scrapped": 1
    }
  ]
}
```

8. NestJS 模块建议

建议最小拆分为以下模块：

- `database`
- `imports`
- `devices`
- `dashboard`

8.1 imports 模块职责

- 上传 Excel
- 解析 Excel
- 字段校验
- 原始表写入
- 标准化写入

8.2 devices 模块职责

- 设备总览查询
- 设备明细查询
- 聚合统计查询

8.3 dashboard 模块职责

- 提供前端总览类接口
- 聚合多个业务模块的数据输出

9. 目录建议

```
apps/api/src
  main.ts
  app.module.ts
  modules/
    dashboard/
      dashboard.controller.ts
      dashboard.service.ts
    devices/
      devices.controller.ts
      devices.service.ts
      dto/
    imports/
      imports.controller.ts
      imports.service.ts
      parsers/
    database/
      prisma/
      repositories/
```

10. 前后端联调约定

前端当前已实现设备总览页，并支持 mock 与真实接口切换。

前端约定如下：

- 页面路由：`/device-overview`
- 数据服务文件：`apps/web/src/lib/api/device-overview.ts`
- 环境变量：
- `NEXT_PUBLIC_API_BASE_URL`
- `NEXT_PUBLIC_USE_MOCK_DATA`

10.1 切换到真实后端的方法

前端 `.env.local` 配置：

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:3001
NEXT_PUBLIC_USE_MOCK_DATA=false
```

若后端未准备好：

```
NEXT_PUBLIC_USE_MOCK_DATA=true
```

11. 后端环境变量建议

后端建议配置：

```
PORT=3001
NODE_ENV=development
DATABASE_URL=postgresql://postgres:password@localhost:5432/asset_bi
CORS_ORIGIN=http://localhost:3000
UPLOAD_DIR=./uploads
MAX_FILE_SIZE_MB=20
```

如果启用 Redis：

```
REDIS_URL=redis://localhost:6379
```

12. 本地部署步骤

12.1 PostgreSQL 准备

1. 安装 PostgreSQL 2. 创建数据库：

```
create database asset_bi;
```

3. 创建业务用户并授权

12.2 安装依赖

```
npm install
```

12.3 配置环境变量

创建 `.env`：

```
PORT=3001
DATABASE_URL=postgresql://postgres:password@localhost:5432/asset_bi
CORS_ORIGIN=http://localhost:3000
UPLOAD_DIR=./uploads
MAX_FILE_SIZE_MB=20
```

12.4 初始化数据库

如果使用 Prisma：

```
npx prisma migrate dev
npx prisma generate
```

12.5 启动后端

```
npm run start:dev
```

13. 云服务器部署建议

如果你准备先把后端提前部署到云服务器，建议采用：

- `Node.js + PM2 + Nginx + PostgreSQL`

13.1 服务器准备

- 安装 Node.js 20+
- 安装 PostgreSQL
- 安装 Nginx
- 安装 PM2

13.2 代码部署

```
git clone <your-repo>
cd apps/api
npm install
```

13.3 生产环境变量

```
PORT=3001
NODE_ENV=production
DATABASE_URL=postgresql://postgres:password@127.0.0.1:5432/asset_bi
CORS_ORIGIN=https://your-frontend-domain.com
UPLOAD_DIR=./uploads
MAX_FILE_SIZE_MB=20
```

13.4 PM2 启动

```
pm2 start dist/main.js --name asset-bi-api
pm2 save
pm2 startup
```

13.5 Nginx 反向代理

示例:

```
server {
  listen 80;
  server_name api.your-domain.com;

  location / {
    proxy_pass http://127.0.0.1:3001;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

14. 首期建议优先实现的后端能力

建议优先顺序如下：

1. Excel 上传接口 2. 原始入库 3. 标准化拆分入库 4. 设备总览四个查询接口 5. 前端联调

不建议一开始就做：

- 通用报表设计器
- 动态图表引擎
- 复杂权限树
- 多租户

15. 当前前端已经准备好的内容

前端已完成：

- Dashboard 中的设备总览跳转入口
- `设备总览` 页面
- Framer Motion 层叠动态卡片 UI
- 设备总览 mock 数据层
- 与真实接口兼容的数据访问层

因此后端只要按本文档接口返回，前端即可快速切换到真实数据。

16. 交付建议

推荐你的落地顺序：

1. 后端先部署基础环境 2. 完成数据库表 3. 打通 Excel 上传与标准化 4. 返回设备总览接口 5. 前端关闭 mock 开关并联调

这样能最快实现“上传真实 Excel 后，设备总览页面自动展示真实数量”的目标。